第三次实验指导

裁决、扩展与综合验收

> 数据完整性裁决 · 模态扩展 · 综合验收

目录

第一部分：实验内容与任务

- 1.1 必要概念
- 1.2 实验内容
- 1.3 实验任务
- 1.4 配套材料

第二部分：实验过程及要求

- 2.1 裁决状态管理
- 2.2 多数判定与重复抑制
- 2.3 透明还原
- 2.4 故障注入与反馈
- 2.5 双向与自动验证
- 2.6 3/4/5 模态扩展（选做）
- 2.7 成果整理与作业提交
- 2.8 代码 TODO 与任务对应
- 2.9 过程记录与完成检查

第三部分：实验指导

- 3.1 建立记录目录并运行裁决模型
- 3.2 合并裁决状态声明
- 3.3 实现目的网关多数状态机
- 3.4 实现透明还原
- 3.5 启动项目并检查无故障场景
- 3.6 加入故障表项
- 3.7 读取统计并生成反馈报告
- 3.8 运行自动场景
- 3.9 反向故障检查（可选）
- 3.10 扩展 MPLS 或地理位置标识（选做）
- 3.11 整理证据并检查最终作业
- 3.12 清理

> 实验主题：基于 P4 的多模态网络内生安全
> 前置条件：已完成模态调度和三种异构封装，并创建 `class2-multicast` tag

本次实验继续使用团队项目，在目的网关完成数据完整性裁决、重复抑制和透明还原，再通过故障、双向和自动场景验证系统。已完成三模态基础的小组，可继续增加 MPLS 或地理位置标识模态。实验结果将用于最终作业；拟评良好或优秀的小组还需据此准备后续课程的小组汇报，具体时间由老师通知。

第一部分：实验内容与任务

1.1 必要概念

1. 数据完整性裁决

数据完整性裁决是对同一份原始数据的多个模态副本进行比较，根据比较结果决定是否输出、输出哪一份，并记录异常模态。裁决发生在目的网关，对主机保持透明：主机发送一份普通 IPv4/UDP 或 IPv4/TCP 报文，目的主机最多收到一份还原后的普通报文。

2. 2/3 多数

三份副本中，任意两个不同模态的内容一致即形成 2/3 多数。在“同一序号中最多一个模态出错”的实验假设下，多数结果可作为输出内容：

三份结果	裁决结果
A、A、A	接受 A，只输出一份
A、A、B	接受 A，B 对应模态记为异常
A、B、B	接受 B，A 对应模态记为异常
A、B、C	没有多数，不输出
A、A、缺失	两份已经一致，可以接受 A

2/3 多数不能解决两个模态同时产生相同错误的情况，CRC32 也不是密码学 MAC。本实验主要研究冗余转发、错误检测和数据平面状态处理。

3. 入口 CRC 与目的网关 CRC

`source_crc32` 是源网关在报文进入多模态路径前计算的参考值。目的网关收到副本后，对实际到达的内层字段重新计算 CRC，本文称为 `observed_crc`。

多数比较使用 `observed_crc`。三份副本携带的 `source_crc32` 可能始终相同，直接比较它们无法反映路径中发生的内容变化。如果小组选择 TCP，源网关和目的网关都要使用本组确定的 TCP 字段范围；选择 UDP 时同理。

4. register、counter 和状态窗口

交换机处理一份报文时产生的普通临时数据，在这份报文离开后不会自动保留。但裁决需要先记住第一份副本的结果，等同一序号的第二份、第三份副本到达后再比较，因此需要使用 `register` 保存少量跨报文信息。

组件	保存什么	在本实验中的作用
`register`	某一组副本当前的处理状态	记住序号、已到达的模态、候选 CRC 和是否已经完成裁决，让后到的副本能与前面的副本比较
`counter`	某类事件累计发生的次数	统计各模态正常、异常以及无多数等结果，供实验检查和反馈使用

简单来说，`register` 回答“这一组报文目前处理到哪一步”，`counter` 回答“这种结果一共出现了多少次”。`counter` 只能累计次数，不能代替 `register` 保存某个序号的裁决过程。

`register` 可以理解成一排数量有限、能够重复使用的储物格，这一排储物格称为“状态窗口”。

- `slot` 是其中一个储物格，用来暂存一组报文的裁决状态。
- `seq_tag` 是储物格上的序号标签，说明格子里当前保存的是哪一个报文序号的状态。

报文到达时，先根据它的序号找到一个 slot，再查看该 slot 的 `seq_tag`：

- 两个序号相同：这是同一组的后续副本，继续使用已有状态进行比较。
- 两个序号不同：这个 slot 中保存的是旧报文状态，先清除旧内容，再写入新序号并开始新的裁决。

因此，`slot` 解决“状态放在哪里”，`seq_tag` 解决“这个状态属于哪个序号”。

5. 候选值、已见模态与决定状态

目的网关可以保存最先出现的一个或两个 CRC 候选值。新副本与某个候选值一致时，就可形成两票。

`seen_modes` 是已经参与该序号裁决的模态位图。它用来防止同一 `modality_id` 的重复副本被当成第二票。`decided` 表示该序号已经形成多数或已确定无多数；它用来防止迟到副本产生第二次主机输出。

6. 首个副本与输出时机

第一份到达的副本可能恰好被篡改。在看到第二个不同模态的一致结果之前，还不能形成多数。P4 流水线也不适合在 register 中无限期保存任意长度的完整报文。一种可行方法是保存前面副本的 CRC 状态，当当前副本形成多数时，直接还原并输出当前副本。

7. 透明还原

透明还原是在多数形成后删除网络内部的外层模态头和 PolyShim，恢复源主机原本发送的报文，再从主机端口输出。透明的含义是两端主机不需要识别多模态网络内部使用的协议头和裁决状态。

8. 故障注入与反馈

故障注入是为测试裁决逻辑人为构造的可重复异常。“篡改”可以对选定数据位做 XOR（按位异或），“丢包”则丢弃某一模态副本。故障应当能按模态和序号定位，并能在下一场测试前清除。

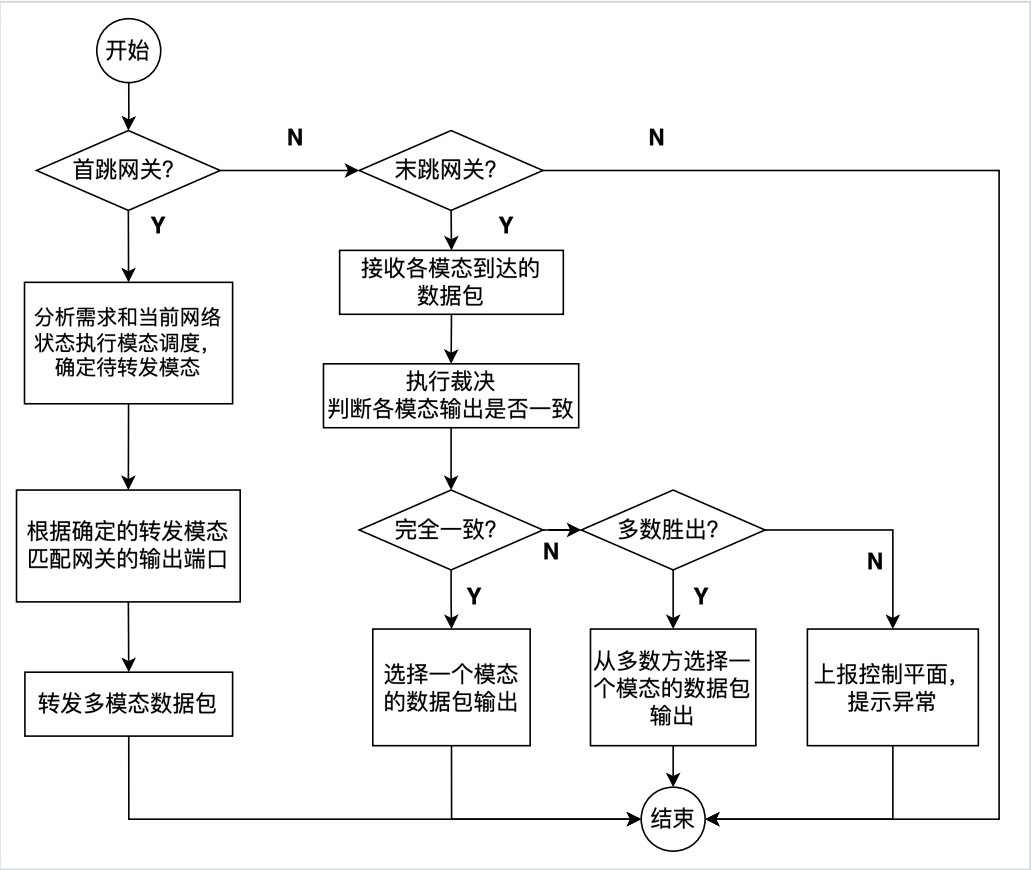
反馈是控制器读取 good、bad 和 no-majority 等统计后，对各模态状态作出评价。三模态基础实现中，反馈先用于报告状态，不宜直接停用某个模态；否则只剩两份副本，无法继续进行 2/3 裁决。

9. 新增模态

三模态是基础内容。新增模态需要有自己的协议头、转发依据和独立路径，且能完成封装、中间转发和目的网关解封装。MPLS 使用 label 作为转发依据；地理位置标识模态可以使用固定长度的区域或节点标识查找下一跳。

四模态或五模态并不意味着每份原始报文都要发送四份或五份副本。为了继续使用 2/3 裁决，调度可以从可用模态中每次选择三个。

10. 完整处理流程



图中源网关负责调度和封装，目的网关负责接收各模态副本、比较结果、选择输出或报告异常。本次实验将这些环节连接起来，并用多个独立场景验证。

1.2 实验内容

在已完成的三模态调度基础上，为目的网关增加按序号管理的裁决状态。目的网关对各模态实际到达的内容计算 CRC，完成 2/3 多数、重复抑制和单一输出，并将胜出副本还原为主机原报文。随后加入可重复的篡改和丢包场景，核对主机结果、路径证据和统计反馈。

进度允许时，可以新增 MPLS 或地理位置标识模态，并尝试四选三或五选三调度。

1.3 实验任务

1. **裁决状态管理**：按报文序号保存有限的裁决状态，并正确处理索引复用。
2. **多数判定与重复抑制**：使不同模态的任意两份一致结果可以形成多数，并避免重复或迟到副本多次输出。
3. **透明还原**：删除网络内部封装，将胜出内容恢复为普通 IPv4/UDP 或 IPv4/TCP 报文。
4. **故障注入与反馈**：构造可定位、可重复的篡改和丢包场景，并记录各模态的裁决统计。
5. **双向与自动验证**：在两个通信方向核对网关角色，并用相互独立的测试场景检查整体结果。
6. **3/4/5 模态扩展（选做）**：新增 MPLS 或地理位置标识模态，并根据实际完成程度接入调度、裁决和统计。
7. **成果整理与作业提交**：整理小组报告、完整源代码、演示视频、实验依据和每名成员的个人材料。

1.4 配套材料

```
第3次实验材料_裁决扩展与综合验收/
├── 第三次实验指导_裁决扩展与综合验收.pdf
├── 代码/
│   ├── 接口示例/
│   ├── 验收工具/
│   ├── 提交模板/
│   └── check-submission.sh
├── 运行结果/
│   ├── verified-adjudication.txt
│   └── verified-final.txt
└── 提交模板/
    ├── 小组报告模板.docx
    └── 个人工作说明与AI使用记录模板.docx
```

路径	用途
`代码/接口示例/adjudication_state.p4.inc`	register 和 counter 声明参考
`代码/接口示例/adjudication_model.py`	可直接运行的 2/3 状态机模型
`代码/验收工具/run-poly-tests.py`	独立启动和检查多个场景
`代码/验收工具/read-counters.py`	读取 good、bad 和 no-majority 统计
`代码/验收工具/evaluate-counters.py`	把统计整理为模态状态报告
`代码/验收工具/send-poly.py`	材料内的 UDP 发送脚本
`代码/验收工具/receive-poly.py`	材料内的 UDP 接收和数量检查脚本

本材料不重复提供调度开发骨架。裁决功能应继续写入团队项目的 `polymorphic.p4` 和 `mycontroller.py`。验收工具默认使用 UDP 参考方案；选择 TCP 的小组可按本组表名、脚本和报文格式调整工具中的适配部分，同时保留原有测试场景和数量判定。

第二部分：实验过程及要求

本次从 `class2-multicast` 之后继续开发。下面说明需要达到的结果，但不限定小组必须使用某一种 register 布局、表名或 action 划分方式。只要方案能在 P4 数据面完成裁决，并且代码、命令和运行证据可以相互对应，就可以采用。

2.1 裁决状态管理

目的网关需要把相同 `sequence_number` 的副本归入同一组，并为同时处理的多组报文保存有限状态。可参考 `slot = sequence_number & 1023` 建立 1024 项状态窗口，也可选择其他窗口大小或映射方法。

无论使用哪种方案，都需要说明：

- 不同序号如何分组；
- 索引发生复用时如何判断是新报文；
- 哪些状态在新报文开始时重置；
- 多个报文连续发送时，前一报文状态不会污染后一报文。

`adjudication_state.p4.inc` 只给出一组可用的声明方式，可以根据小组状态机设计调整。

2.2 多数判定与重复抑制

每份副本到达目的网关后，对本组选定的内层字段计算 `observed_crc`。多数应由不同模态的实际到达内容形成，不直接比较三份 `source_crc32`。

状态机需要处理以下情况：

- 前两份内容一致；
- 前两份不同，第三份与其中一份一致；
- 三份内容都不同；
- 某一模态重复到达；
- 多数形成后，第三份副本才到达；
- 三种模态以不同顺序到达。

实现时可以使用两个 CRC 候选、模态位图和决定标记，也可以使用能达到同等结果的其他定长状态结构。验证时不应假定模态 1、2、3 按固定顺序到达，也不应让同一模态的重复副本形成两票。

2.3 透明还原

只有在当前副本使多数成立时，才将它还原后送往主机。还原后的报文应与源主机原本发送的传输层协议一致：UDP 方案恢复 IPv4/UDP，TCP 方案恢复 IPv4/TCP。目的主机不应看到 PolyShim、外层模态头或重复报文。

具体的 header 赋值和 invalidate 顺序可以根据小组报文结构确定，但需要同时处理 Ethernet EtherType、内层 IPv4、目的主机 MAC、出口端口和相关 checksum。

2.4 故障注入与反馈

故障可以放在三条路径的中间交换机，使异常发生在多模态转发域内，而不是直接改动主机输入。材料内参考控制器使用以下参数：

```
--fault corrupt:MODE:SEQ:MASK
--fault drop:MODE:SEQ
```

小组可以使用其他表名或参数形式，但故障需要能定位到某一模态和某一序号，同一设置可以重现，并且在 clean 后不会留在下一场景。

统计至少能区分各模态的正常、异常和无多数事件。材料中的 `evaluate-counters.py` 把 bad>0 记为 `SUSPECT`，有 good 且无 bad 记为 `HEALTHY`，没有样本记为 `NO_DATA`。小组也可以设计其他评价方法，需在报告中说明规则和运行结果。

2.5 双向与自动验证

基础验证至少包含以下五个相互独立的场景：

场景	需要检查的结果
正向无故障	h1 发送一份，h2 收到正确一份
反向无故障	S2 调度、S1 裁决，h1 收到正确一份
单模态篡改	未篡改的两份形成多数，主机仍收到原文一份
单模态丢包	到达的两份一致时仍能输出一份
三份各不相同	主机收到 0 份，无多数统计增加

每个场景建议独立完成 clean、启动、安装故障、发送、检查输出、读取统计和清理，避免上一场景的表项、register、counter 或序号影响当前结果。

`run-poly-tests.py` 是与材料 UDP 参考实现配套的测试骨架。如果本组使用 TCP，或调整了表名、计数器名，可以修改对应的收发和接口适配部分，但保留这五个场景及其数量和内容判定。

2.6 3/4/5 模态扩展（选做）

加分项从三模态基础功能正常的版本开始。可以使用预留的 `s41-s42` 路径增加 MPLS 或地理位置标识；如果同时实现两种，第五模态需要有另一条可区分的路径。

一个新模态需要能说明：

1. 协议头字段和转发依据；
2. 源网关封装、中间节点转发和目的网关解封装；
3. 与其他模态可区分的路径；
4. 目的主机收到还原后的 IPv4/UDP 或 IPv4/TCP 报文；
5. 代码、控制器表项、pcap 和收发结果；
6. 新增模态后，三模态基础的五个场景仍然正常。

只改 EtherType、`modality_id` 或普通 IPv4 目的地址，没有新的协议处理和独立路径，不能作为一个完整的新模态。

扩展可按完成程度逐步进行：先完成新模态独立收发，再将它接入每包三选的调度，最后接入 2/3 裁决、故障注入和统计。不同小组可以根据方案选择完成层级。

2.7 成果整理与作业提交

最终作业由组长统一提交 ZIP 压缩包，文件名为：

```
组号-组长学号-组长姓名.zip
```

压缩包参考结构：

```
组号-组长学号-组长姓名.zip
├──── 小组报告-组号-组长学号-组长姓名.docx
├──── 小组报告-组号-组长学号-组长姓名.pdf
├──── 演示视频-组号-组长学号-组长姓名.mp4
├──── codes/
│    ├──── README.md
│    └──── ......                  # P4、控制器、拓扑、配置和测试脚本
├──── 个人材料/
│    ├──── 个人总结-组号-学号-姓名.docx
│    └──── ......                  # 每名成员各一份
└──── evidence/
     ├──── evidence-index.md
     └──── ......                 # 截图、pcap、日志、统计和测试输出
```

- 小组报告同时保留可编辑 DOCX 和导出的 PDF。
- `codes/` 中放本组最终实现的完整源代码，不只放代码截图。`codes/README.md` 说明环境、启动命令、测试命令、主要文件和最终 Git commit。
- 演示视频使用 MP4 格式，H.264 编码，时长不超过 5 分钟，文件不超过 500 MB。视频中至少展示项目或关键代码位置、一个正常场景和一个故障场景。
- 每名成员使用 `个人工作说明与AI使用记录模板.docx` 独立填写一份个人材料。
- `evidence-index.md` 把报告中的主要结论与截图、pcap、日志、统计或自动测试输出逐项对应。
- 无需提交虚拟机镜像，也无需放入可重新生成的 `build/`、`logs/` 和 `pcaps/` 等大体积目录；需要作为证据的个别文件可选取后放入 `evidence/`。

小组报告应能说明：副本如何关联，调度、封装和裁决位于哪些代码位置，CRC 覆盖哪些字段，篡改、丢包和无多数时分别发生什么，如何处理重复、迟到和索引复用，当前方案的能力和限制，以及各成员的实际工作与贡献比例。

这里的“小组报告”是最终作业中的书面材料，各组均需提交。小组汇报采用分档要求：

汇报时间：由老师通知，在后续课程中统一进行。
汇报对象：小组汇报是评定良好或优秀的必要环节；不参加汇报的小组，成绩等级最高为中等。拟评及格或中等的小组不要求参加汇报。

需要汇报的小组使用最终报告、项目代码和实验结果说明实际完成情况，可展示无故障、故障、反向验证及扩展模态证据，不另行撰写重复的汇报材料。

2.8 代码 TODO 与任务对应

本次继续使用项目中的 `STUDENT_TODO.md`，主要处理 M5 和 M6。如果项目中缺少该文件，可从上次材料的 `代码/调度开发骨架/STUDENT_TODO.md` 补回。

TODO 与文件	参考内容	对应任务
`polymorphic.p4`：Ingress `TODO M5`	识别目的网关流量，读取和更新序号、候选 CRC、模态位图、决定和统计状态	裁决状态管理；多数判定与重复抑制
`polymorphic.p4`：metadata `TODO M3-M6`	增加 register 读值、当前 slot、候选值、裁决结果和还原所需临时字段	裁决状态管理；透明还原
`polymorphic.p4`：Deparser `TODO M6`	对胜出副本只序列化还原后的 Ethernet、IPv4、UDP 或 TCP 和数据	透明还原
`mycontroller.py`：控制面 TODO	为 S1、S2 配置源网关和目的网关角色，并支持故障表项	故障注入与反馈；双向与自动验证
`代码/验收工具/`	根据本组的表名、计数器名和 UDP/TCP 选择进行适配	故障注入与反馈；双向与自动验证

2.9 过程记录与完成检查

`class3/` 是小组在团队 Git 仓库中建立的阶段记录目录，不会由 Python 模型或 P4 项目自动生成。可参考：

```
class3/
├── state-design.txt
├── member-progress.txt
└── evidence/
```

`state-design.txt` 可以用来说明第一份为什么不输出、序号与 slot 如何对应、前两份不同时第三份如何决定结果、如何防止重复模态和多次输出。`member-progress.txt` 记录状态设计、P4 实现、故障构造、测试和结果分析的实际分工。

提交前可检查：

- [ ] 裁决位于目的网关 P4 数据面，不是由主机脚本直接选结果。
- [ ] 到达顺序变化时，任意两个不同模态的一致结果都能形成多数。
- [ ] 同一模态的重复副本不能形成第二票，迟到副本不会再次输出。
- [ ] 无故障、单模态篡改、单模态丢包和无多数场景的主机结果正确。
- [ ] 正向和反向的调度、裁决角色可以互换。
- [ ] 透明还原后，主机不会看到外层模态头和 PolyShim。
- [ ] 新功能完成后，上次实验的三条封装路径和原始 Demo 功能仍然正常。
- [ ] 小组报告、完整源代码、演示视频、实验依据和全体成员个人材料齐全。
- [ ] 最终 ZIP 文件名与目录结构符合要求。
- [ ] 如拟评良好或优秀，已准备后续课程的小组汇报和成员回答。

功能稳定后可以分别创建 `class3-adjudication` 和 `class4-final` tag，前者记录裁决功能完成的版本，后者指向与最终报告和演示视频一致的提交版本。

第三部分：实验指导

下列命令与材料内 UDP 参考实现配套。选择 TCP 的小组可沿用同样的实验顺序和预期报文数量，将其中的收发脚本、Parser、CRC 和 checksum 处理替换为本组 TCP 方案。

3.1 建立记录目录并运行裁决模型

在团队 Git 仓库根目录建立本阶段记录：

```
cd <团队Git仓库根目录>
mkdir -p class3/evidence
touch class3/state-design.txt class3/member-progress.txt
```

运行材料内状态机模型：

```
cd <本课目录>/代码/接口示例
python3 adjudication_model.py all
```

也可分别运行：

```
python3 adjudication_model.py no-fault
python3 adjudication_model.py corrupt-mode1
python3 adjudication_model.py drop-mode2
python3 adjudication_model.py no-majority
```

模型只展示状态转移，并对 16 字节 `protected_data` 计算 CRC。最终功能需要写入目的网关 P4 数据面，并按本组 UDP 或 TCP 方案对实际到达的内层字段重新计算 CRC。

模型参考输出：

```
SCENARIO no-fault
mode=2 value=correct action=DROP first-candidate
mode=1 value=correct action=ACCEPT majority
mode=3 value=correct action=DROP already-decided
OUTPUT count=1

SCENARIO corrupt-mode1
mode=1 value=wrong action=DROP first-candidate
mode=2 value=correct action=DROP second-candidate
mode=3 value=correct action=ACCEPT majority
OUTPUT count=1

SCENARIO drop-mode2
mode=1 value=correct action=DROP first-candidate
mode=3 value=correct action=ACCEPT majority
OUTPUT count=1

SCENARIO no-majority
mode=1 value=value-a action=DROP first-candidate
mode=2 value=value-b action=DROP second-candidate
mode=3 value=value-c action=DROP no-majority
OUTPUT count=0
```

3.2 合并裁决状态声明

对照 `adjudication_state.p4.inc`，在项目 `polymorphic.p4` 中加入所需 register 和 counter，并在 metadata 中加入 register `read()` 的临时读值。可按下列顺序逐步编译：

```
slot 与 seq_tag 初始化
-> seen_modes
-> CRC 候选和候选数量
-> decided 与胜出 CRC
-> good / bad / no-majority 统计
```

每完成一步执行：

```
make build
```

编译成功后，再用连续两个不同序号和会复用同一 slot 的序号检查初始化逻辑。

3.3 实现目的网关多数状态机

控制器需要让 S1 和 S2 能根据通信方向切换角色：

```
h1 -> h2: S1 调度，S2 裁决
h2 -> h1: S2 调度，S1 裁决
```

中间交换机不进入目的网关裁决分支。目的网关的处理流程可参考：

```
读取当前 slot
-> seq_tag 不同时初始化
-> 检查当前 modality_id 是否已经出现
-> 对本次实际内容计算 observed_crc
-> 与已有候选比较
-> 形成多数时设置输出标记
-> 无多数、重复或迟到时丢弃
```

为避免实现只对一种偶然到达顺序有效，可在 Python 模型中调换模态输入顺序，再在 P4 运行中根据 pcap 的实际到达顺序检查。

3.4 实现透明还原

当当前副本形成多数时，可执行与报文结构对应的还原 action。UDP 参考实现需要检查：

1. Ethernet EtherType 恢复为 IPv4；
2. 内层 IPv4 恢复为最终 IPv4 header；
3. 外层 IPv4、IPv6、Source Routing 和 PolyShim 不再发出；
4. 目的 MAC 和主机出口与方向一致；
5. IPv4 checksum 重新计算；
6. `decided` 之后到达的副本只用于统计，不再输出。

TCP 方案还需要根据本组是否更改 TCP 头或数据检查 TCP checksum。完成后执行：

```
make build
python3 -m py_compile mycontroller.py
```

3.5 启动项目并检查无故障场景

终端 A：

```
cd <实验项目目录>
make
```

终端 B：

```
cd <实验项目目录>
python3 mycontroller.py
```

材料 UDP 参考方案中，先在 h2 启动接收：

```
mininet> h2 sh -c 'python3 receive-poly.py --timeout 10 --expect 1 > /tmp/h2-poly.log 2>&1 &'
```

再从 h1 发送一份：

```
mininet> h1 python3 send-poly.py 10.0.2.2 class3-ok
mininet> h2 cat /tmp/h2-poly.log
```

预期只有一条 `POLY_RECV`，并出现：

```
POLY_SUMMARY received=1 expected=1
```

反向检查：

```
mininet> h1 sh -c 'python3 receive-poly.py --timeout 10 --expect 1 > /tmp/h1-poly.log 2>&1 &'
mininet> h2 python3 send-poly.py 10.0.1.1 reverse-one
mininet> h1 cat /tmp/h1-poly.log
```

选择 TCP 的小组在这里使用本组收发脚本，继续使用“发送一份，正确接收一份”的数量判定。

3.6 加入故障表项

材料参考方案在三条路径的第一台中间交换机安装故障：

模态	中间交换机	故障对象
1	s11	IPv4 模态副本
2	s21	IPv6 模态副本
3	s31	Source Routing 模态副本

单模态篡改示例：

```
python3 mycontroller.py --fault corrupt:1:1:0x1
```

按无故障方式收发。预期未篡改的模态 2、3 形成多数，主机收到原文一份，模态 1 的 bad 统计增加。

单模态丢包示例：

```
python3 mycontroller.py --fault drop:2:1
```

预期模态 1、3 形成多数，主机收到一份。未到达的模态 2 没有数据包事件，因此材料模型不会仅凭本次裁决自动将它记为 bad。

无多数示例：

```
python3 mycontroller.py \
  --fault corrupt:1:1:0x1 \
  --fault corrupt:2:1:0x2 \
  --fault corrupt:3:1:0x4
```

接收端的期望数量改为 0：

```
mininet> h2 sh -c 'python3 receive-poly.py --timeout 4 --expect 0 > /tmp/h2-none.log 2>&1 &'
mininet> h1 python3 send-poly.py 10.0.2.2 no-majority
mininet> h2 cat /tmp/h2-none.log
```

预期 `received=0`，并且 `no_majority` 增加。每次改变故障设置前，退出 Mininet，执行 `make stop` 和 `make clean`，再重新启动当前场景。

3.7 读取统计并生成反馈报告

将 `代码/验收工具/` 中的以下文件复制到实验项目根目录：

```
read-counters.py
evaluate-counters.py
send-poly.py
receive-poly.py
run-poly-tests.py
```

语法检查：

```
python3 -m py_compile \
  mycontroller.py send-poly.py receive-poly.py \
  read-counters.py evaluate-counters.py run-poly-tests.py
```

项目运行后读取统计：

```
python3 read-counters.py
python3 read-counters.py | python3 evaluate-counters.py
```

材料参考报告示例：

```
GATEWAY s2
mode1 status=SUSPECT good=0 bad=1
mode2 status=HEALTHY good=1 bad=0
mode3 status=HEALTHY good=1 bad=0
no_majority status=CLEAR count=0
schedule_policy=ALL_THREE_MODES
```

如果本组修改了计数器名或评价方法，同步调整读取脚本，并在记录中说明实际规则。

3.8 运行自动场景

先退出手工启动的 Mininet 并清理：

```
make stop
make clean
```

再执行：

```
python3 run-poly-tests.py
```

材料 UDP 参考实现的目标输出：

```
SCENARIO_PASS no-fault
SCENARIO_PASS reverse
SCENARIO_PASS corrupt-mode1
SCENARIO_PASS drop-mode2
SCENARIO_PASS no-majority
ALL_SCENARIOS_PASS 5
```

单独调试某个场景：

```
python3 run-poly-tests.py no-fault
python3 run-poly-tests.py corrupt-mode1
python3 run-poly-tests.py no-majority
```

如果本组选择 TCP 或使用不同接口，先完成测试工具适配，再保留五个场景运行。不用删除场景、放宽接收数量或写死输出的方式获得通过标记。

材料参考实现的关键结果：

场景	主机结果	统计特征
无故障	`received=1`	三个模态 good=1
模态 1 篡改	`received=1` 且内容正确	mode1 bad=1，mode2/3 good=1
模态 2 丢包	`received=1`	mode1/3 good=1
三份各不同	`received=0`	no_majority=1
反向无故障	h1 `received=1`	S1 记录模态统计

更完整的输出见 `运行结果/verified-adjudication.txt` 和 `运行结果/verified-final.txt`。

3.9 反向故障检查（可选）

内置五个场景中的 `reverse` 是反向无故障。如果进度允许，可再手工检查一个反向篡改场景。在 clean 状态下启动：

```
python3 mycontroller.py --fault corrupt:1:1:0x1
```

Mininet：

```
mininet> h1 sh -c 'python3 receive-poly.py --timeout 10 --expect 1 > /tmp/h1-reverse-fault.log 2>&1 &'
mininet> h2 python3 send-poly.py 10.0.1.1 reverse-bad
mininet> h1 cat /tmp/h1-reverse-fault.log
```

预期 h1 收到正确内容一份，S1 的模态 1 bad 增加，模态 2、3 good 增加。

3.10 扩展 MPLS 或地理位置标识（选做）

先保存三模态五个场景通过的 commit，再开始扩展。建议使用预留路径：

```
h1-s1-s41-s42-s2-h2
```

MPLS 方案参考

```
const bit<16> TYPE_MPLS = 0x8847;

header mpls_t {
    bit<20> label;
    bit<3>  tc;
    bit<1>  bos;
    bit<8>  ttl;
}
```

单层 label 可设 `bos=1`。如果使用 label stack，Parser 需要解析至 `bos=1`，再进入 PolyShim 和内层报文。处理流程可设计为：

```
源网关：压入 MPLS label，设置 EtherType 0x8847
s41/s42：根据 label 执行 forward 或 swap
目的网关：弹出 MPLS，进入已有裁决和透明还原流程
```

正向和反向可以使用不同 label，具体数值由小组设计。验证时检查 EtherType、label、`bos`、`ttl`、s41/s42 路径和目的网关还原结果。

地理位置标识方案参考

可以使用本地实验 EtherType，并设计固定长度 header。一种参考定义为：

```
const bit<16> TYPE_GEO = 0x88b5;

header geo_t {
    bit<16> src_region;
    bit<16> dst_region;
    bit<16> current_node;
    bit<16> next_node;
    bit<8>  hop_limit;
    bit<8>  flags;
}
```

区域和节点编号可由小组定义，但需要固定且能够解释，不将可变长字符串直接放入 P4 header。处理流程可设计为：

```
源网关：写入源/目的区域和第一跳节点
s41/s42：根据区域或节点字段查找下一跳
每跳：按设计更新节点字段，hop_limit 减 1
目的网关：删除地理位置头，进入裁决和透明还原流程
```

验证时检查实验 EtherType、各字段、s41/s42 路径、逐跳变化和最终还原结果。

四选三与五选三

四模态可以组织以下三模态组合：

```
G123 = IPv4 + IPv6 + Source Routing
G124 = IPv4 + IPv6 + 新模态
G134 = IPv4 + Source Routing + 新模态
G234 = IPv6 + Source Routing + 新模态
```

五模态不必配置全部十种三模态组合，但可以使五个模态都至少出现在一个可选组合中，并用不同序号或控制器策略产生多种实际组合。无论可用模态总数是多少，每份原始报文仍只产生三份用于 2/3 裁决的副本。

完成新模态后，重新运行三模态基础的五个场景。若原有场景失败，先检查新增 Parser、端口、模态位图、组合选择和裁决统计范围。

3.11 整理证据并检查最终作业

先运行材料内自检脚本：

```
bash <本课目录>/代码/check-submission.sh <实验项目目录>
git status --short
git log --oneline --decorate -8
```

再手工核对：

1. `codes/README.md` 中的启动和测试命令能否从 clean 状态重现；
2. 报告、演示视频、代码和 evidence 是否对应同一个 Git commit；
3. 每个主要结论是否能找到命令、pcap、日志、统计或截图依据；
4. 演示视频的时长、大小、编码和文件名是否符合要求；
5. 每名成员的个人材料是否已放入 `个人材料/`；
6. 如拟评良好或优秀，小组汇报所需的演示顺序、证据和成员回答是否已准备。

确认最终 commit 后可创建：

```
git tag class4-final
```

若 tag 已存在，先确认指向，不覆盖已有标记。

3.12 清理

```
mininet> exit
```

```
make stop
make clean
```